

Order Management System

This document outlines the 5-week assignment roadmap for building a single full-fledged Order Management System.

The learning topics are already covered separately. This document strictly defines weekly implementation deliverables aligned with the mentorship learning plan.

Each week builds progressively on the same project.

Backend and Frontend effort should remain balanced (50/50 overall).

Week 1 – Database Design & Core CRUD Implementation

- Design ER Diagram for Order Management System.
- Entities: Users, Products, Orders, OrderItems.
- Implement Code-First models with proper relationships (1-M, M-M where applicable).
- Configure DbContext and create initial migration.
- Seed initial product data.
- Implement basic CRUD APIs for Products and Orders.
- Use DTOs instead of exposing entities directly.
- Frontend: Build Product listing page and Create Order form.
- Connect frontend with backend using Axios.

Week 2 – LINQ Queries & API Structuring

- Refactor APIs with proper routing and DTO responses.
- Implement LINQ-based data retrieval for Orders with total amount calculation.
- Add filtering by date range and user.
- Add pagination to Orders listing endpoint.
- Ensure projection before materialization (avoid unnecessary Includes).
- Frontend: Implement pagination UI.
- Add filtering UI components.
- Introduce basic loading and error handling states.

Week 3 – Clean Architecture Refactor

- Restructure backend using Layered Architecture (Controller, Service, Repository, DTO, Mapping).
- Implement Dependency Injection properly.
- Integrate AutoMapper for DTO mapping.
- Implement global exception handling middleware.
- Add advanced queries: Aggregated order summary, product sales summary.
- Frontend: Introduce Redux Toolkit.
- Manage Order state using Redux slices.

Week 4 – Authentication & Security Implementation

- Implement User Registration and Login APIs.
- Generate JWT tokens with claims.
- Implement role-based authorization (Admin/User).
- Secure order creation and modification endpoints.
- Implement password hashing.
- Frontend: Implement login page.
- Store JWT securely and implement protected routes.
- Integrate RTK Query for authenticated API calls.

Week 5 – Performance, Microservice & Background Jobs

- Optimize queries using AsNoTracking and projection.
- Add database indexing where required.
- Implement transaction handling during order creation.
- Introduce Logging using ILogger or Serilog.
- Create Inventory Microservice with separate database.
- Implement service-to-service communication using HttpClientFactory.
- Validate stock before order confirmation.
- Implement Background Job (Hangfire or BackgroundService) to process order confirmation or notification.
- Frontend: Handle stock validation errors gracefully.
- Implement RTK Query advanced features (cache invalidation, optimistic updates).

Final Submission Requirements

- Clean Architecture structure.
- Proper DTO usage.
- JWT secured APIs.
- Pagination and filtering implemented.
- Microservice communication working locally.
- Background job functioning.
- Basic unit tests for service layer.
- Professional frontend structure with Redux Toolkit and RTK Query.